

Task 1: Mobile App Development Process.

- i. Types of Mobile applications.
- ii. Mobile apps programming languages.
- iii. Mobile app development frameworks.
- iv. Mobile app architectures and design patterns.
- v. Requirement engineering process.
- vi. Mobile app development cost estimation.

Mobile App Development.

Mobile app development is the process of creating software applications for mobile devices like smartphones, tablets, and digital assistants. With the world increasingly relying on mobile technology, apps have become essential for communication, entertainment, productivity, and more.

Developing a mobile app involves using specialized tools and programming languages, such as Swift for iOS or Java for Android, to build apps that are user-friendly and functional.

Mobile app development covers several areas:

- **User Experience (UX) and User Interface (UI) Design:** Creating an app that's easy to use and visually appealing.
- **Backend Development:** Building the server-side systems that power the app, like databases or cloud services.
- **Quality Assurance (QA):** Testing the app to ensure it works smoothly without bugs.

The Mobile App Development Process.

Creating a mobile app involves a series of steps to ensure the final product is functional, user-friendly, and successful. Here's a detailed look at each stage:

1. Conceptualization

This is the brainstorming phase where developers and teams come up with ideas for the app. The goal is to identify:

- **Unique Value Proposition:** What makes the app special or different from others?
- **Target Audience:** Who will use the app, and what problems will it solve for them?
- **Key Features:** What functions or capabilities will the app have?

For example, if you're creating a fitness app, you might decide it should track workouts, offer diet tips, and connect users with trainers. This stage lays the foundation for the entire project.

2. Planning and Analysis

Once the idea is clear, it's time to plan the project. This involves:

- **Defining the Scope:** Deciding what features will be included in the first version of the app.

- **Market Research:** Studying competitors and user preferences to ensure the app meets market demands.
- **Project Plan:** Creating a timeline, budget, and list of resources needed.
- **User Stories and Prototypes:** Writing user stories (e.g., “As a user, I want to log my meals easily”) and creating wireframes or basic prototypes to visualize the app’s structure.

This stage ensures everyone involved understands the app’s goals and how to achieve them.

3. Design

The design phase focuses on creating the app’s look and feel. This includes:

- **User Interface (UI) Design:** Choosing colors, fonts, icons, and layouts to make the app visually appealing.
- **User Experience (UX) Design:** Ensuring the app is easy to navigate and intuitive to use. For example, buttons should be easy to tap, and menus should be simple to understand.
- **Prototyping:** Building interactive mockups to test the design before coding begins.

Good design is crucial because it determines how users interact with and perceive the app.

4. Development

This is where the app starts to come to life through coding.

Development is split into two main parts:

- Frontend Development: Writing code for the app’s interface—what users see and interact with. For iOS, developers use Swift or Objective-C; for Android, they use Kotlin or Java.
- Backend Development: Building the server-side systems that store data, manage user accounts, or connect to third-party services (e.g., payment gateways or social media APIs).

Developers may also integrate features like push notifications or in-app purchases during this stage.

5. Testing

Testing ensures the app works as intended and is free of issues. This involves:

- **Functionality Testing:** Checking if all features work correctly (e.g., does the login button work?).
 - **Usability Testing:** Ensuring the app is easy to use for the target audience.
 - **Performance Testing:** Verifying the app runs smoothly, even on older devices or slow networks.
 - **Bug Fixing:** Identifying and resolving errors, crashes, or glitches.
- Testing is critical to delivering a polished app that users will enjoy.

6. Deployment

Once the app is tested and ready, it's submitted to app stores like Apple's App Store or Google Play. This involves:

- **Preparing the App:** Creating store listings, screenshots, and descriptions that follow app store guidelines.
- **Submission Process:** Uploading the app and waiting for approval (which can take a few days to weeks, depending on the platform).
- **Launch:** Making the app available for users to download.

Developers must ensure the app meets all store requirements to avoid rejection.

7. Maintenance

After launch, the app needs ongoing care to stay relevant and functional:

- **Bug Fixes:** Addressing issues reported by users.
- **Updates:** Adding new features or improving existing ones based on user feedback.
- **Compatibility:** Ensuring the app works with new device models or operating system updates.

Regular maintenance keeps users happy and helps the app stay competitive.

I. Types of Mobile applications.

Mobile applications are software programs designed to run on smartphones and tablets. They help users perform specific tasks such as learning, messaging, shopping, entertainment, and more. When testing

these apps with Appium (Appium is an open-source, cross-platform test automation framework for mobile (and some desktop/TV) apps. It allows you to automate using the W3C WebDriver protocol), it's important to know the different types of mobile applications.

Below are the four major types of mobile apps used in Appium testing.

1. Native apps

These are designed for specific mobile operating systems like Android, IOS, or Windows. Here, Android is developed using Java and IOS uses Swift or Objective C. They are published on Play Store or App Store. **Examples:** WhatsApp, Google Maps, Spotify, Telegram

Technologies used: C++, Java, Kotlin, Objective-C, Python, React Native, Swift.

Advantages:

1. Fast and smooth performance
2. Highly secure
3. Works well with device hardware (camera, GPS, sensors)

Disadvantages:

1. Separate development for each platform
2. Higher development and maintenance cost

2. Web Apps

Mobile web applications are optimized for mobile devices and they are accessible through a web browser on a smartphone. They runs with the help of web browser applications like chrome, safari, Firefox etc. **Examples:** Amazon (mobile site), Canva, Netflix (web), Walmart

Technologies used: C#, C++, HTML, CSS, Javascript, Java, Perl, PHP, Ruby, Python, Typescript.

Advantages:

1. Easy to build and maintain
2. Works on all devices
3. Low development cost

Disadvantages:

1. Browser dependent
2. Slower than native apps
3. Limited access to device features

3. Hybrid apps

Hybrid apps combine elements of both native and web apps, using a single codebase to run on multiple platforms. These are deployed on container that uses mobile WebView object. **Examples:** Instagram, Gmail, LinkedIn, Twitter, Uber

Technologies Used: HTML5, CSS, Flutter, JavaScript, Appy Pie, AppMakr, Ionic, Objective-C, Swift, React Native, Wrap-Up.

Advantages:

1. Faster development
2. Single codebase for all platforms
3. Access to device features (camera, GPS)

Disadvantages:

1. Slower than native apps
2. Less interactive and smooth

4. Progressive Web Apps (PWA)

PWAs are web apps that provide a native app like experience. These are designed and developed similar to web apps. These apps take support of services workers. These are published on play store or app store. **Examples:** Pinterest, Starbucks, Tinder, Trivago.

Technologies used: AngularJS, CSS, HTML5, JavaScript, Lighthouse, Polymer, PWA library, VueJS, WebAssembly.

Advantages:

1. Works on any device
2. Fast and responsive
3. Can work offline
4. Cheaper to build and maintain

Disadvantages:

1. Limited hardware access
2. Cannot use features like Bluetooth or NFC reliably

II. Mobile apps programming languages.

1. Java
2. Kotlin
3. C++
4. C#

5. Python
6. HTML, CSS, JavaScript
7. Dart
8. Corona

III. Mobile app development frameworks.

There are Several leading Mobile App Development Frameworks that can help you to build Mobile Applications more efficiently and conveniently:

1. React Native

React Native is one of the most recommended Mobile App Frameworks in the development industry. The framework, created by Facebook, is an open-source framework that offers you to develop mobile applications for Android & iOS platforms. The React Native framework is based on React and JavaScript that aims to develop native applications over hybrid applications that run on a web view. Some of the major benefits of React Native are mentioned below:

- Code Re-usability & Cost-Effective
- Compatible with third-party plugins
- Re-usable components for optimal performance
- Provides hot deployment features
- Ease of Maintenance

There are various renowned mobile applications built with React Native such as Instagram.

2. Flutter

Flutter, developed by Google, is a UI toolkit to build native applications for mobile apps, desktop & web platforms. Flutter is a cross-platform mobile app development framework that works on one code base to develop Android as well as iOS applications. Some of the major benefits of Flutter are mentioned below:

- Provides Full Native Performance
- Flexible User interface (UI)
- Provides Strong Widget Support
- Offers Built-in Material Design
- Fast Application Development

Some of the major mobile applications that are built on Flutter are - Google Ads, SpaceX Go.

3. Ionic

Ionic, developed in 2013, is an open-source framework that allows you to build cross-platform for mobile apps using web technologies like HTML, CSS & JavaScript. The application built through the Ionic framework can work on Android, iOS and Windows platforms. Moreover, Ionic has its own command-line interface. Some of the major benefits of Ionic for mobile development apps are mentioned below:

- Faster Application Development
- Availability of Cordova Plugins
- Built-in UI components
- Platform Independent
- Based on AngularJS

Various renowned mobile applications are built with Ionic such as Amtrak.

4. Xamarin

Xamarin is also one of the most popular open-source frameworks used to develop mobile applications. The framework, acquired by Microsoft, is based on .Net and allows you to build native applications for Android iOS and Windows platforms. There are various benefits of Xamarin, some of these are mentioned below:

- Huge Community of around 1.4 million developers
- Native API Access & UI Support
- Easier API Integration
- Target All Platforms
- Cost-Effective & Faster Development Process

Some of the most popular and renowned mobile applications that are built on Xamarin are Alaska Airlines.

5. Mobile Angular UI

Mobile Angular UI is an open-source mobile application development framework that merges the implementation of

the Bootstrap and Angular frameworks. The framework offers various UI components such as overlays, switches , sidebars, navbars, to provide robust and attractive user interfaces for the applications. Also supports fastclick.js and overthrow.js to create rich experiences. Some of the major benefits of Mobile Angular UI for mobile development apps are mentioned below:

- Responsive Framework
- Offers Interactive Components
- Lightweight
- No jQuery dependencies
- Easier conversion from the web or desktop application to the mobile app

There are various renowned mobile applications built with Mobile Angular UI such as iKeyBox.

6. Adobe PhoneGap

PhoneGap is among the most popular mobile app development frameworks. PhoneGap, purchased by Adobe Systems in 2011, is an open-source framework that allows you to build cross-platform applications using languages such as HTML5, CSS3, and JavaScript. The framework allows you to create a single mobile application that can be installed as a native app over many devices. There are various benefits of PhoneGap, some of these are mentioned below:

- Code Re-usability
- No Hardware Restrictions
- Plugin Library & Third-Party tools
- Robust Support System
- Compatibility with all the platforms

Some of the most popular and renowned mobile applications that are built on PhoneGap for mobile development apps are - Wikipedia.

7. Appcelerator Titanium

Titanium, developed by Appcelerator Inc, is an open-source development framework that allows you to build mobile apps for all

platforms. The framework is based on a Javascript-based SDK with over 5000 APIs for Windows, HTML5, iOS, and Android that further allows the reusability of Javascript code up to 90% in case you're targeting multiple platforms for application development. There are various benefits of the Titanium SDK, some of these are mentioned below:

- Various Features for API Management
- Compatible with multiple platforms
- Easier API Integration
- Better Code Reusability
- Faster Development Process

There are various renowned mobile applications built with Titanium SDK such as Massachusetts Institute of Technology (MIT).

8. Framework 7

Framework7 is an open-source development framework that allows the development of mobile, desktop, or web applications with a native perspective. The framework allows you to use tools of your preferences with HTML, CSS, & JavaScript and offers the additional support of Vue.js and React components for app development. There are various benefits of Framework 7, some of these are mentioned below:

- Easier Customization
- Provides various UI widgets and elements
- Paired with additional tools like Electron & NW.js
- Rich Ecosystem
- Ease of Maintenance

Various renowned mobile applications are built with Framework 7 such as Wappler.

9. JQuery Mobile

JQuery Mobile is a cross-platform development framework that is used to build mobile and web applications for various devices. The framework allows running a single code version across all devices. Furthermore, the HTML5-based development framework comes with various plugins such as image slider and content-slider that help to create a feature-rich and interactive design. There are various benefits of JQuery Mobile, some of these are mentioned below:

- Responsive Framework

- Supports multiple platforms & devices
- Lightweight Size
- Various Add-on Plugins
- Powerful Theming Framework

Some of the significant mobile applications that are built on JQuery Mobile are Veev.

10. NativeScript

Native Script is an open-source framework that is used to build native mobile applications using Angular, Vue.js, TypeScript, or JavaScript. The framework allows you to create both Android and IOS apps from a single codebase. The apps built with the NativeScript framework use the same APIs as in Xcode or Android Studio. Some of the major benefits of NativeScript are mentioned below:

- Cross-Platform Development
- Direct access to Android & iOS APIs
- Native Performance
- Re-usability of Code
- Strong Backend Support

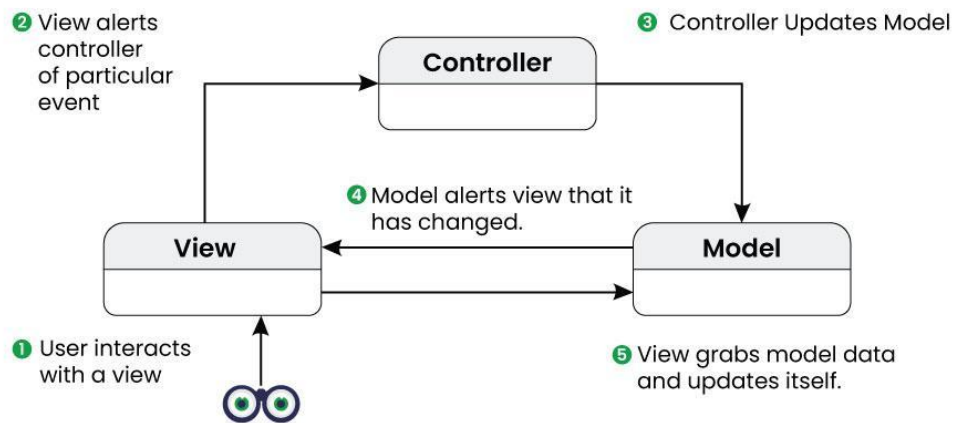
Some of the significant mobile applications that are built on NativeScript are PUMA.

IV. Mobile app architectures and design patterns.

Architectures.

Mobile architectures(behind-the-scenes structure that turns code into the awesome apps), like blueprints, are used to make sure everything fits and works well. It turns your idea into a smooth and working app. Below are several architectures being used:

➤ Classic MVC or MVC (Model-View-Controller) Architecture



Classic MVC, or Model-View-Controller, is an architectural pattern used in software development to organize the code of an application in a structured way. Let's break down each part of MVC:

- **Model:**
 - Think of the Model as the brain of your application. It manages the data and the rules for how the data can be changed. For example, if you're building a to-do list app, the Model would handle tasks, deadlines, and any other information related to the to-do items.
- **View:**
 - The View is like the face of your app. It's what the user sees and interacts with. In our to-do list example, the View would be the part of the app where you see your list of tasks, checkboxes, and maybe a button to add new tasks.
- **Controller:**
 - The Controller is the middleman between the Model and the View. It takes input from the user (like clicking a button to add a new task), updates the Model based on that input, and then tells the View to refresh and show the updated information. In our to-do list, the Controller would handle things like adding or deleting tasks.

When to use Classic MVC Architecture

Use Classic MVC when your app is not too big or too small – it's like the Goldilocks choice for moderate-sized projects. If you want a clear separation between how data is handled (Model), how it's shown to the user (View), and how the user interacts with it (Controller), then Classic MVC is a good fit. It's a balanced approach that works well for many different types of applications.

➤ **Apple's MVC Architecture**

Apple's MVC (Model-View-Controller) architecture is a specific implementation of the classic MVC pattern tailored for iOS app development. Let's break down Apple's MVC in simple terms:

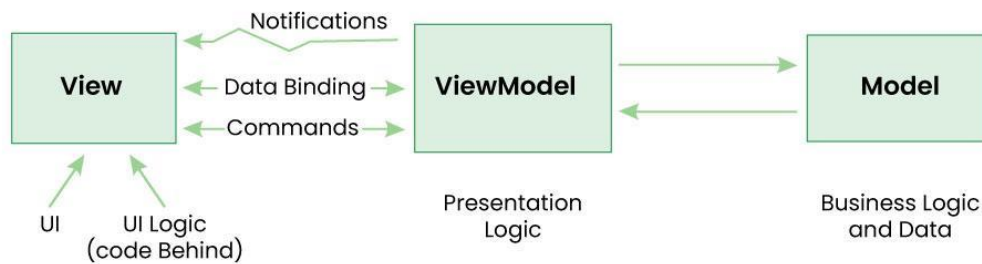
- **Model:**
 - Just like in the classic MVC, the Model in Apple's MVC is responsible for managing the data and business logic of your app. For example, in a note-taking app, the Model would handle tasks such as saving and retrieving notes.
- **View:**
 - The View is what the user interacts with on the screen. In an iOS app, this could be a button, a label, or any other interface element that users can see and touch. In the note-taking app, the View would be where you read and edit your notes.
- **Controller:**
 - The Controller acts as a coordinator between the Model and the View. It takes user input from the View, updates the Model accordingly, and manages the communication between the two. In the note-taking app, the Controller would handle actions like creating a new note or deleting an existing one.

When to use Apple's MVC Architecture

- Use Apple's MVC when you're developing an iOS app. It's the default architecture recommended by Apple for building apps on their platform.
- If you're comfortable working within the iOS development ecosystem and want a structure that aligns with Apple's guidelines, Apple's MVC is a straightforward and effective choice.
- It works well for a wide range of iOS applications, from simple utilities to more complex applications.

➤ **MVVM (Model-View-View-Model) Architecture**

MVVM, or Model-View-View Model, is an architectural pattern commonly used in software development, especially for building user interfaces. Let's break it down:



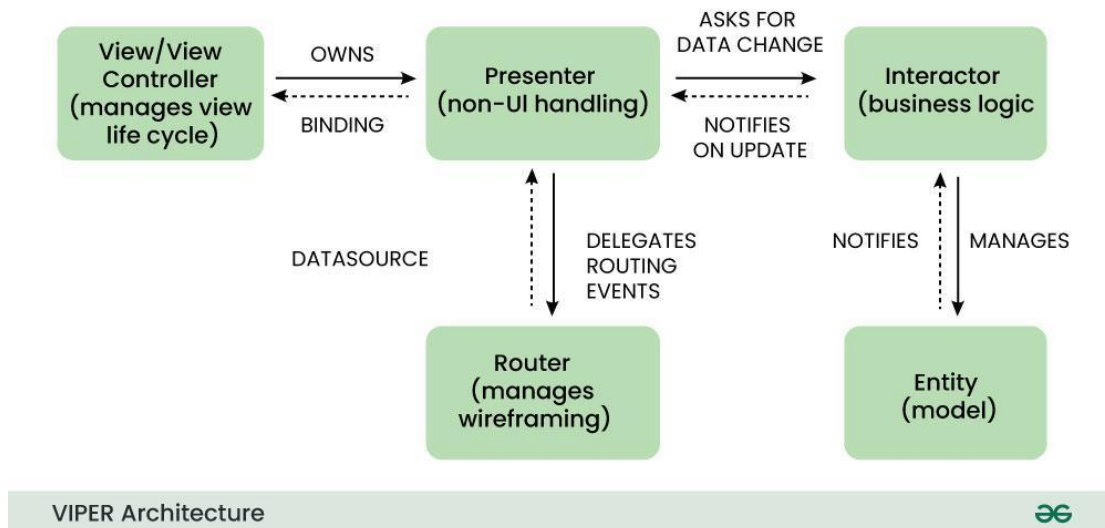
- **Model:**
 - The Model is responsible for managing the data and the business logic of your application. If you're building a weather app, for example, the Model would handle things like fetching weather data from the internet and storing it.
- **View:**
 - The View is what the user sees on the screen – buttons, text, images, etc. In a weather app, the View would be the part of the app where you see the current temperature, weather conditions, and a forecast.
- **View-Model:**
 - The View-Model is like a bridge between the Model and the View. It takes data from the Model and prepares it in a way that the View can easily display. It also handles user interactions and updates the Model accordingly. In the weather app, the View Model would take the raw weather data from the Model, format it, and then provide it to the View for display.

When to use MVVM Architecture

Use MVVM when you want a structured way to organize your code and keep a clear separation between how your data is managed (Model), how it's presented to the user (View), and the logic that connects the two (View-Model). MVVM is particularly effective when used with data-binding frameworks, as it simplifies the synchronization between the UI and the underlying data. It's a good choice for projects with complex user interfaces or applications where data presentation and manipulation are crucial.

➤ VIPER (View Interactor Presenter Entity Router) Architecture

VIPER is an architectural pattern used in software development, especially in mobile app development, to organize code in a modular and scalable way. Let's break down:



- **View:**

- The View is what the user sees on the screen – buttons, text, images, etc. In a messaging app, for instance, the View would be the part of the app where you see your chat interface.

- **Interactor:**

- The Interactor contains the business logic of your application. It's responsible for handling data and performing operations. In a messaging app, the Interactor would manage tasks like sending and receiving messages.

- **Presenter:**

- The Presenter acts as a mediator between the View and the Interactor. It handles user input from the View, communicates with the Interactor to perform actions, and then updates the View with the results. In the messaging app, the Presenter would manage things like handling a user's tap on a message to display more details.

- **Entity:**

- The Entity represents the data objects used by the Interactor. In the messaging app, an Entity could be the structure that holds information about a message, such as the sender, timestamp, and content.

- **Router:**

- The Router manages navigation between different modules (screens or features) of your app. It decides which screen to

show based on user actions. In the messaging app, the Router could handle navigating from the chat screen to the settings screen.

When to use VIPER Architecture

- Use VIPER when you are working on a large and complex project, especially if you want to keep your code modular and easily maintainable.
- VIPER is suitable for applications with multiple features that need to be developed and modified independently.
- It's a good choice when you have a team of developers working on different parts of the app or when scalability and maintainability are top priorities.

Design Patterns .

Singleton Method Design Pattern

The singleton policy ensures that there is only one instance of a class and provides global access. This is especially useful when you want to manage a single instance of an object or control access to a delayed object.

Factory Method Design Pattern

The Factory Method model defines an interface for creating an object but allows subclasses to modify the type of the created object.

Especially useful when you need to create objects with a common interface but different functionality.

Observer Method Design Pattern

The observer structure defines one to many dependencies between objects, so when one object changes its state, all its dependents are automatically notified and updated. This is useful for scheduling distributed events.

Dependency Injection (DI) Method Design Pattern

Dependency Injection is a method of providing class dependencies from the outside, rather than creating them in the class. It improves code modularity and testability by making classes independent of their dependencies.

Adapter Method Design Pattern

The adapter configuration allows you to use the interface of an existing class as a link to a new one. It is often used to work with others without modifying the source code of existing classes.

Strategy Method Design Pattern

The strategy model defines a family of algorithms, contains each of them, and provides them with flexibility. It allows you to select the appropriate algorithm at runtime. This example is useful when you want to provide different options for a task.

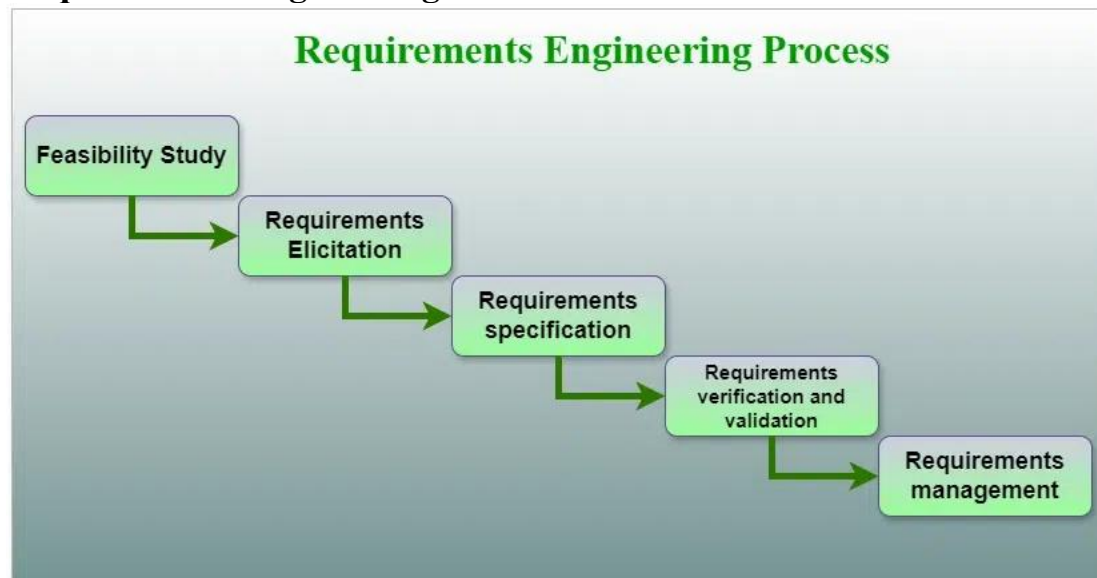
Composite Method Design Pattern

A composite pattern allows you to arrange objects in a tree structure to represent a part-of-the-whole structure. This is helpful when you have to deal with individual objects and sets of objects accurately.

V. Requirement engineering process.

Requirements Engineering is the process of identifying, eliciting, analyzing, specifying, validating, and managing the needs and expectations of stakeholders for a software system.

Requirements Engineering Process



Requirements Engineering Process

1. Feasibility Study

The feasibility study mainly concentrates on below five mentioned areas below. Among these Economic Feasibility Study is the most important

part of the feasibility analysis and the Legal Feasibility Study is less considered feasibility analysis.

1. **Technical Feasibility:** In Technical Feasibility current resources both hardware software along required technology are analyzed/assessed to develop the project. This technical feasibility study reports whether there are correct required resources and technologies that will be used for project development. Along with this, the feasibility study also analyzes the technical skills and capabilities of the technical team, whether existing technology can be used or not, whether maintenance and up-gradation are easy or not for the chosen technology, etc.
2. **Operational Feasibility:** In Operational Feasibility degree of providing service to requirements is analyzed along with how easy the product will be to operate and maintain after deployment. Along with this other operational scopes are determining the usability of the product, Determining suggested solution by the software development team is acceptable or not, etc.
3. **Economic Feasibility:** In the Economic Feasibility study cost and benefit of the project are analyzed. This means under this feasibility study a detailed analysis is carried out will be cost of the project for development which includes all required costs for final development hardware and software resources required, design and development costs operational costs, and so on. After that, it is analyzed whether the project will be beneficial in terms of finance for the organization or not.
4. **Legal Feasibility:** In legal feasibility, the project is ensured to comply with all relevant laws, regulations, and standards. It identifies any legal constraints that could impact the project and reviews existing contracts and agreements to assess their effect on the project's execution. Additionally, legal feasibility considers issues related to intellectual property, such as patents and copyrights, to safeguard the project's innovation and originality.
5. **Schedule Feasibility:** In schedule feasibility, the project timeline is evaluated to determine if it is realistic and achievable. Significant milestones are identified, and deadlines are established to track progress effectively. Resource availability is assessed to ensure that the necessary resources are accessible to meet the project schedule. Furthermore, any time constraints that might affect project delivery are considered to ensure timely completion. This focus on schedule feasibility is crucial for the successful planning and execution of a project.

2. Requirements Elicitation

Requirements elicitation is the process of gathering information about the needs and expectations of stakeholders for a software system. This is the first step in the requirements engineering process and it is critical to the success of the software development project. The goal of this step is to understand the problem that the software system is intended to solve and the needs and expectations of the stakeholders who will use the system.

Several techniques can be used to elicit requirements, including:

- **Interviews:** These are one-on-one conversations with stakeholders to gather information about their needs and expectations.
- **Surveys:** These are questionnaires that are distributed to stakeholders to gather information about their needs and expectations.
- **Observation:** This technique involves observing the stakeholders in their work environment to gather information about their needs and expectations.
- **Prototyping:** This technique involves creating a working model of the software system, which can be used to gather feedback from stakeholders and to validate requirements.

3. Requirements Specification

Requirements specification is the process of documenting the requirements identified in the analysis step in a clear, consistent, and unambiguous manner. This step also involves prioritizing and grouping the requirements into manageable chunks.

The goal of this step is to create a clear and comprehensive document that describes the requirements for the software system. This document should be understandable by both the development team and the stakeholders.

Several types of requirements are commonly specified in this step, including

1. **Functional Requirements:** These describe what the software system should do. They specify the functionality that the system must provide, such as input validation, data storage, and user interface.
2. **Non-Functional Requirements:** These describe how well the software system should do it. They specify the quality attributes of the system, such as performance, reliability, usability, and security.

3. **Constraints:** These describe any limitations or restrictions that must be considered when developing the software system.
4. **Acceptance Criteria:** These describe the conditions that must be met for the software system to be considered complete and ready for release.

4. Requirements Verification and Validation

Requirements verification and validation (V&V) is the process of checking that the requirements for a software system are complete, consistent, and accurate and that they meet the needs and expectations of the stakeholders. The goal of V&V is to ensure that the software system being developed meets the requirements and that it is developed on time, within budget, and to the required quality.

1. Verification is checking that the requirements are complete, consistent, and accurate. It involves reviewing the requirements to ensure that they are clear, testable, and free of errors and inconsistencies. This can include reviewing the requirements document, models, and diagrams, and holding meetings and walkthroughs with stakeholders.
2. Validation is the process of checking that the requirements meet the needs and expectations of the stakeholders. It involves testing the requirements to ensure that they are valid and that the software system being developed will meet the needs of the stakeholders. This can include testing the software system through simulation, testing with prototypes, and testing with the final version of the software.

5. Requirements Management

Requirement management is the process of analyzing, documenting, tracking, prioritizing, and agreeing on the requirement and controlling the communication with relevant stakeholders. This stage takes care of the changing nature of requirements. It should be ensured that the SRS is as modifiable as possible to incorporate changes in requirements specified by the end users at later stages too. Modifying the software as per requirements in a systematic and controlled manner is an extremely important part of the requirements engineering process.

VI. Mobile app development cost estimation.

Cost estimation determines the scope and resources required for the project.

1. It involves assessing the various aspects of the project to estimate the effort, time, cost, and resources needed to complete the project.
2. Accurate cost estimation is important for effective and efficient project planning, management, and execution.

Importance of Cost Estimation

Here are some of the reasons why cost estimation is critical in project management:

1. **Financial Planning:** Cost estimation helps in planning the financial aspects of the project, thus helping to avoid financial shortfalls.
2. **Resource Planning:** It ensures the necessary resources are identified and allocated accordingly.
3. **Timeline Creation:** It facilitates the development of realistic timelines and milestones for the project.
4. **Identifying Risks:** It helps to identify potential risks associated with overall project execution.
5. **Detailed Planning:** It helps to create a detailed plan for the project execution, ensuring all the aspects of the project are considered.
6. **Planning Quality Assurance:** It helps in planning quality assurance activities and ensuring that the project outcomes meet the required standards.